

使用 Genkit Go 和 Nano Banana Pro 建構相片修復應用程式

來源：<https://codelabs.developers.google.com/cloud-genkit-go-nano-banana?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將瞭解如何使用 Genkit Go、Nano Banana Pro (Gemini 3 Pro Image) 和 Google Cloud Run，建立相片修復指令列應用程式和網頁伺服器。

目錄

1. [1. 簡介](#)
2. [2. 環境設定](#)
3. [3. 專案設定](#)
4. [4. 建立第一個 Genkit 應用程式](#)
5. [5. 使用 Nano Banana Pro 復原圖像](#)
6. [6. 將 glowUp 部署為網路服務](#)
7. [7. 結論](#)

1. 簡介

在本程式碼研究室中，我們將建構相片修復工具 **GlowUp**。GlowUp 會運用 AI 技術修復舊相片、受損相片或黑白相片，並生成高畫質 4K 彩色圖片。你可以使用這項工具讓家庭相片煥然一新，甚至還能修復損壞的插圖、繪圖、油畫或其他形式的圖片。

您將使用 **Genkit Go** 實作應用程式邏輯，並使用 **Gemini 3 Pro Image** (又稱 **Nano Banana Pro**) 做為處理相片的模型。

必要條件

- 具備 Go 程式設計語言基本知識
- Google Cloud 控制台基本知識

課程內容

- 如何使用 Go 開發 Genkit 應用程式
- Genkit 基本概念，例如流程、外掛程式和提示詞
- 如何使用 Handlebars 範本撰寫提示
- 如何從模型回覆取得圖片資料

軟硬體需求

本研討會可完全在 **Google Cloud Shell** 中進行，該環境已預先安裝所有必要依附元件 (gcloud CLI、程式碼編輯器、Go、Gemini CLI)。

或者，如果您偏好使用自己的電腦，則需要下列項目：

- Go 工具鍊 (1.24 以上版本)
- Node.js v20 以上版本 (適用於 `genkit` CLI)
- 已安裝 `gcloud` CLI 的終端機
- 用於編輯程式碼的 IDE，例如 VS Code 或類似工具
- 建議：使用程式設計代理，例如 **Gemini CLI** 或 **Antigravity**

主要技術

請參閱下列資源，進一步瞭解我們會用到的技術：

- Gemini Nano Banana Pro (Gemini 3 Pro Image)：用於復原程序的模型
 - Genkit Go：用於自動調度模型呼叫的工具包
-

步驟 2 · 約 10 分鐘

2. 環境設定

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

請選擇下列其中一個選項：如要在自己的電腦上執行本程式碼研究室，請選取「自行設定環境」；如要完全在雲端執行本程式碼研究室，請選取「啟動 Cloud Shell」。

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots of the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form with a blue box highlighting the 'Project ID' field, which contains the value 'inbound-analogy-389614'. A blue arrow points from the 'NEW PROJECT' button to the 'Project ID' field.

Google Cloud Select a project

Select a project

NEW PROJECT

Search projects and folders

Google Cloud

New Project

Project name *
My Project 13475

Project ID inbound-analogy-389614. It cannot be changed later. EDIT

Location *
BROWSE

Parent organization or folder

CREATE CANCEL

- 專案名稱是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- 專案 ID 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

- 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Cloud Shell interface. The browser tab is titled 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and displays the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 3 · 約 5 分鐘

3. 專案設定

建立專案

首先，我們需要為專案建立新目錄，並初始化 Go 模組。在終端機中執行下列指令：

```
mkdir -p glowup && cd glowup
go mod init glowup
```

安裝 Genkit CLI

現在需要安裝 Genkit CLI。這樣您就能存取本機開發人員工具，包括開發人員 UI。在終端機視窗中輸入：

```
curl -sL cli.genkit.dev | bash
```

設定環境變數

重要事項！ Gemini 3 Pro Image 模型 (`gemini-3-pro-image-preview`) 目前僅在 Vertex AI 的全球位置提供。你必須將位置資訊設為 `global`，還原程序才能運作。

請確認您已設定正確的 Google Cloud 憑證。將 `your-project-id` 替換為實際專案 ID。使用 Gemini 3 Pro 預先發布版模型 (Nano Banana Pro 也是其中之一) 時，必須啟用位置資訊。 `global`

```
export GOOGLE_CLOUD_PROJECT=$(gcloud config get project)
export GOOGLE_CLOUD_LOCATION=global
```

在 Shell 模式下執行下列指令，啟用 Vertex AI API：

```
gcloud services enable aiplatform.googleapis.com
```

如果您是從本機電腦執行 (而非 Cloud Shell)，則需要使用 `gcloud` 指令進行驗證：

```
gcloud auth application-default login
```

4. 建立第一個 Genkit 應用程式

Genkit 是開放原始碼框架，可協助開發人員建構、部署及監控可用於正式環境的 AI 輔助應用程式。在本節中，我們會建立簡單的「Hello World」應用程式，協助您熟悉架構，再深入探討相片修復邏輯。

Genkit 術語

開始使用 Genkit 前，請務必瞭解幾個重要術語：

- **外掛程式**：用於擴充 Genkit 的功能。您可透過外掛程式註冊 AI 模型，為應用程式提供支援。
- **Flow**：Genkit 的主要架構元件。一般流程會接收輸入內容、處理內容，然後傳回輸出內容。不一定需要使用模型，但大多數時候，您會在流程中使用模型。
- **提示詞**：以 `dotprompt` 格式儲存的互動範本 (儲存為 `*.prompt` 檔案)。這些檔案不僅包含模型指令，也包含模型名稱、模型參數、輸入和輸出等設定。

連結至 AI 模型

Genkit 會使用外掛程式，將程式碼連結至模型供應商。所有主要模型供應商 (包括 Google、Anthropic 和 OpenAI) 都有外掛程式。您也可以使用外掛程式連線至本機模型 (例如使用 Ollama)，或擴充 Genkit 的功能 (例如連線至 MCP 伺服器)。

如要存取 Google 模型，請使用 `googlegenai` 外掛程式。支援兩個後端：

- Google AI：最適合用於原型設計。使用 API 金鑰。
- Vertex AI (Google Cloud)：建議用於正式環境。使用專案 ID 和位置。

在本程式碼研究室中，我們將使用 Vertex AI 驗證，參考您在本實驗室一開始建立的專案。

建立迎賓流程

在深入瞭解 Glow Up 相片修復流程之前，我們先建立基本流程，熟悉相關概念，並確保設定正常運作。

Flow 是特殊的 Genkit 函式，可包裝 AI 邏輯，提供下列功能：

- **型別安全輸入和輸出**：使用 Go 結構體定義結構定義，進行靜態和執行階段驗證
- **串流支援**：串流部分回應或自訂資料
- **整合開發人員 UI**：使用視覺化追蹤記錄測試及偵錯流程
- **輕鬆部署**：部署為 HTTP 端點至任何平台

開啟 IDE，並在專案目錄中建立 `main.go` 檔案。如果使用 Cloud Shell，可以執行下列指令：

```
cloudshell edit main.go
```

然後新增下列程式碼：

main.go

```

package main

import (
    "context"
    "log"
    "os"
    "os/signal"
    "syscall"

    "github.com/firebase/genkit/go/genkit"
    "github.com/firebase/genkit/go/ai"
    "github.com/firebase/genkit/go/plugins/googlegenai"
)

func main() {
    ctx, cancel := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    // Initialize Genkit with the Vertex AI plugin
    g := genkit.Init(ctx,
        genkit.WithPlugins(&googlegenai.VertexAI{}),
    )

    // Define the greeter flow
    genkit.DefineFlow(g, "greeter", func(ctx context.Context, name string) (string,
error) {
        text, err := genkit.GenerateText(ctx, g,
            ai.WithModelName("vertexai/gemini-2.5-pro"),
            ai.WithPrompt("Say a warm and creative hello to %s", name),
        )
        if err != nil {
            return "", err
        }

        return text, nil
    })

    // Register the flow here in the next steps
    log.Println("GlowUp initialized. Ready for flows.")
    <-ctx.Done()
}

```

儲存檔案，然後執行 `go mod tidy` 更新依附元件：

```
go mod tidy
```

上述程式碼會使用 VertexAI 外掛程式初始化 Genkit、定義名為「greeter」的流程，然後在 `<-ctx.Done()` 上無限期等待。我們暫緩執行，以免這個程式立即終止，因為我們沒有提供任何實際執行流程的指令。

也就是說，如果您現在執行這個程式，程式本身不會執行太多動作，我們需要以某種方式叫用流程。在實際的正式版應用程式中，我們會將這個流程包裝在網路伺服器或 CLI 應用程式中，但在開發期間，我們可以透過 `genkit` CLI 協助開發及最佳化流程。

開發 `genkit` CLI 的目的是協助我們測試 Genkit 堆疊的模型、提示、流程和其他元件。此外，這項工具也完全支援追蹤，方便您瞭解應用程式的幕後運作方式。如要使用 `genkit` CLI 啟動問候流程，請執行下列指令：

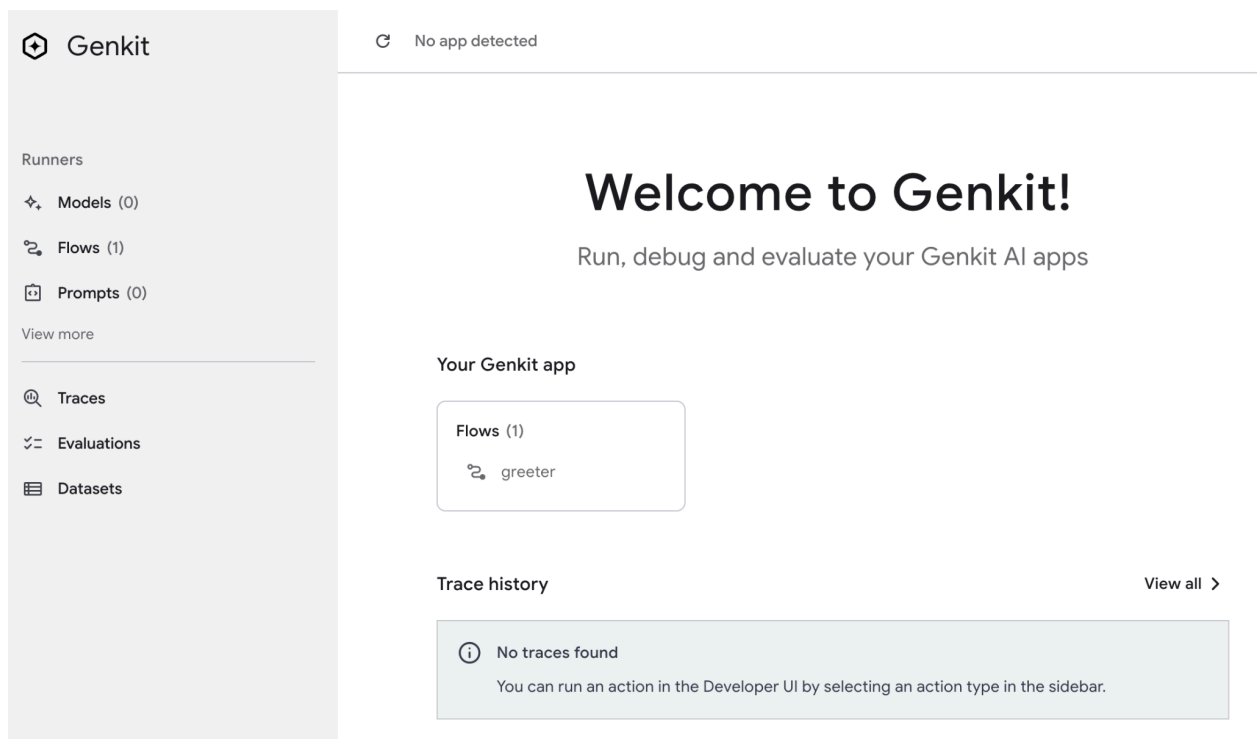
```
genkit start -- go run main.go
```

這會啟動 Telemetry API 和開發人員 UI 端點。畫面應如下所示：

```
$ genkit start -- go run main.go
Telemetry API running on http://localhost:4033
Project root: /home/daniela/glowup
Genkit Developer UI: http://localhost:4000
```

注意：如果您是透過 CloudShell 執行本程式碼研究室，則存在**錯誤**，導致開發人員 UI 無法正確啟動流程。您還是可以使用 `genkit` CLI 執行流程，但要改用指令列。如要這麼做，請在 **Cloud Shell** 中開啟**第二個終端機**，然後執行 `genkit flow:run greeter "name"`。指令

在瀏覽器中開啟 `localhost:4000` 即可啟動開發人員使用者介面。這時畫面上應會顯示類似下方的訊息：



請花點時間探索開發人員 UI。或許可以觸發「問候」流程一次，瞭解運作方式。

使用提示範本

雖然您可以像上一個範例一樣，在模型呼叫中硬式編碼提示，但更好的做法是建立提示範本，集中儲存應用程式的所有提示。這樣不僅方便您在維護程式碼時尋找提示，還能獨立於流程測試提示。

如要定義提示範本，Genkit 會使用開放原始碼的 `dotprompt` 格式，並儲存為 `*.prompt` 檔案。`.prompt` 檔案包含兩個部分：

1. **前言**：定義模型、模型參數，以及輸入和輸出結構定義的 YAML 區塊
2. **內文**：提示本身的內文，可使用「handlebars」語法建立範本。舉例來說，如果您定義名為 `variable-name` 的輸入內容，則可在主體中以 `{{variable-name}}` 參照該內容。

注意：根據預設，Genkit 會根據啟動二進位檔的位置，載入 `prompts` 資料夾中的所有提示詞。您也可以可以在 Genkit 初始化期間自訂這個資料夾名稱。

專案的目錄結構如下所示：

```
glowup/
├── main.go
└── prompts/
    └── greeter.prompt
```

我們來看看實際的運作情形。請先建立資料夾來儲存提示：

```
mkdir -p prompts
```

接著建立 `greeter.prompt` 檔案：

```
cloudshell edit prompts/greeter.prompt
```

並插入下列內容：

注意！由於程式碼研究室平台有其限制，因此無法正確算繪 `*.prompt` 檔案，請點選下方的 GitHub 連結，以文字形式查看檔案的完整內容。

greeter.prompt

```
---
model: vertexai/gemini-2.5-flash
config:
  temperature: 1.9
input:
  schema:
    name: string
---
{{ role "system" }}
You are a member of the world renowned greeters guild.
You take pride in making the best greetings ever made.
Greet the user saying something positive to brighten their day.

{{ role "user" }}
Hi! I'm {{ name }}, nice to meet you!
```

這個提示詞展示了範本語言的幾項功能。首先，我們在 frontmatter 中指定模型 `vertexai/gemini-2.5-flash`。如要指定模型，請使用相關外掛程式說明文件中定義的命名方式。

我們可以在設定區段中設定模型參數。我們使用溫度參數 1.9，讓模型更具創意。溫度參數範圍介於 0 (輸出內容更一致) 到 2 (輸出內容更具創意)。這項和其他模型參數通常會發布在模型的模型資料表中。舉例來說，以下是 [gemini-2.5-flash](#) 的模型資訊頁面。

輸入區段可讓我們指定提示的引數。在本例中，我們將名稱定義為字串。在 frontmatter 之後，就是提示詞主體。第一個區塊定義系統提示，第二個區塊則是使用者提示。這些元素加總起來，可實現非常強大的提示技術。如要查看 `dotprompt` 的完整說明文件，請按[這裡](#)。

現在，讓我們調整 `greeter` 流程，使用剛才建立的提示：

[main.go](#)

```

package main

import (
    "context"
    "os"
    "os/signal"
    "syscall"

    "github.com/firebase/genkit/go/ai"
    "github.com/firebase/genkit/go/genkit"
    "github.com/firebase/genkit/go/plugins/googlegenai"
)

func main() {
    ctx, cancel := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    // Initialize Genkit with the Vertex AI plugin
    g := genkit.Init(ctx,
        genkit.WithPlugins(&googlegenai.VertexAI{}),
        genkit.WithDefaultModel("vertexai/gemini-2.5-flash"),
    )

    // Define the greeter flow
    genkit.DefineFlow(g, "greeter", func(ctx context.Context, name *string) (string,
error) {
        prompt := genkit.LookupPrompt(g, "greeter")

        input := map[string]any{
            "name": name,
        }

        resp, err := prompt.Execute(ctx, ai.WithInput(input))
        if err != nil {
            return "", err
        }
        return resp.Text(), nil
    })

    <-ctx.Done()
}

```

請嘗試透過指令列執行流程，並使用及不使用名稱，看看有什麼差異。不使用名稱執行：

```
genkit flow:run greeter
```

輸出內容範例：

```
$ genkit flow:run greeter
Telemetry API running on http://localhost:4035
Running '/flow/greeter' (stream=false)...
Result:
"Hello there, absolutely delightful human!\n\nThe very moment your message arrived,
the day instantly sparkled a little brighter. It's not just nice, it's genuinely
**wonderful** to meet you!\n\nMay your entire day be filled with unexpected pockets
of joy, effortless triumphs, and all the happiness you truly deserve! We're thrilled
to have you here!"
```

並提供名稱：

```
genkit flow:run greeter '{"name":"Daniela"}'
```

輸出內容範例：

```
$ genkit flow:run greeter '{"name":"Daniela"}'
Telemetry API running on http://localhost:4035
Running '/flow/greeter' (stream=false)...
Result:
"Well hello there, Daniela! What a truly beautiful name, and what an absolute
pleasure it is to meet you!\n\nRight from this moment, I just know your day is going
to be brimming with positive energy and wonderful surprises. May it be filled with
brilliant ideas, joyful moments, and the delightful realization that you're an
amazing person doing incredible things. So glad you're here!"
```

您會發現範本運作正常。我們已準備好讓這項專案更上一層樓！

5. 使用 Nano Banana Pro 復原圖像

還原提示

您已熟悉 Genkit 的建構區塊，現在可以建立相片修復提示。

在提示目錄中建立名為 `glowup.prompt` 的檔案，並貼上以下內容：

注意！由於程式碼研究室平台有其限制，因此無法正確算繪 *.prompt 檔案，請點選下方的 GitHub 連結，以文字形式查看檔案的完整內容。

glowup.prompt

```
---
model: vertexai/gemini-3-pro-image-preview
config:
  imageConfig:
    imageSize: "4K"
input:
  schema:
    url: string
    contentType: string
---

{{role "system"}}
You are GlowUp, a professional-grade photo restorer.
Your goal is to provide a "surgical" restoration service that transforms
vintage, damaged, or monochrome photographs into high-fidelity 4K colorized
versions.

RULES:
1. **Grounding**: You are strictly grounded in the original source pixels. Do
NOT add new objects (trees, people, buildings, etc.) that are not present in
the source. Additionally, do NOT remove any elements from the source, unless
they are clearly defects that do not belong in the original scene.
2. **Fidelity**: Preserve the original facial expressions and identity of
subjects. Do NOT "beautify" or alter features in a way that changes the
person's identity.
3. **Background**: Preserve background fidelity. Overexposed light sources
(like windows) must remain as light sources. Do not "fill in" missing details
with invented scenery.
4. **Colorization**: If the image is monochrome, colorize it realistically,
respecting historical accuracy where possible.
5. **Upscaling**: Output a high-fidelity image.

{{role "user"}}
Restore this photo.
Image: {{ media url=url contentType=contentType }}
```

還原提示與「迎賓」提示的模式相同，但有幾項明顯差異：

- **圖片大小**：`imageConfig` 的 `imageSize` 屬性是 Nano Banana Pro 的模型專屬參數。可讓我們將輸出大小指定為 1K、2K 或 4K。

- **媒體輸入：**我們使用 `{{ media }}` 範本，將相片插入使用者提示。這項技術可讓我們將多模態提示詞傳送至模型 (文字 + 圖片)。

您可以在開發人員 UI 中測試這個提示。歡迎隨意調整，看看對輸出內容有何影響。

圖片還原流程

還原提示準備就緒後，現在來建構 CLI 應用程式。將 `main.go` 的內容替換為下列程式碼：

[main.go](#)

```

package main

import (
    "context"
    "encoding/base64"
    "errors"
    "flag"
    "fmt"
    "log"
    "mime"
    "os"
    "os/signal"
    "strings"
    "syscall"

    "github.com/firebase/genkit/go/ai"
    "github.com/firebase/genkit/go/genkit"
    "github.com/firebase/genkit/go/plugins/googlelegenai"
)

func main() {
    url := flag.String("url", "", "url of the image to restore")
    contentType := flag.String("contentType", "image/jpeg", "content type of the image
(default: image/jpeg)")
    flag.Parse()

    ctx, cancel := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
    defer cancel()

    // Initialize Genkit with the Vertex AI plugin
    g := genkit.Init(ctx,
        genkit.WithPlugins(&googlelegenai.VertexAI{}),
    )

    // Input schema for the glowUp flow
    type Input struct {
        URL      string `json:"url,omitempty"`
        ContentType string `json:"contentType,omitempty"`
    }

    glowup := genkit.DefineFlow(g, "glowUp", func(ctx context.Context, input Input)
(string, error) {
        // 1. Retrieve prompt
        prompt := genkit.LookupPrompt(g, "glowup")
        if prompt == nil {
            return "", errors.New("prompt 'glowup' not found")
        }

        resp, err := prompt.Execute(ctx, ai.WithInput(input))
        if err != nil {
            return "", fmt.Errorf("generation failed: %w", err)
        }
    })
}

```



```

    }

    return resp.Media(), nil
})

// triggers the flow and returns the encoded response from the model
out, err := glowup.Run(ctx, Input{URL: *url, ContentType: *contentType})
if err != nil {
    log.Fatalln(err)
}

// decodes image data and returns the appropriate file extension
data, ext, err := decode(out)
if err != nil {
    log.Fatalln(err)
}

// writes restored file to disk
filename := "restored" + ext
if err := os.WriteFile(filename, data, 0644); err != nil {
    log.Fatalln(err)
}
}

// decode returns the decoded data and the file extension appropriate for the mime type
func decode(text string) ([]byte, string, error) {
    if !strings.HasPrefix(text, "data:") {
        return nil, "", errors.New("unsupported encoding format")
    }
    text = strings.TrimPrefix(text, "data:")
    parts := strings.Split(text, ";base64,")

    mimeType := parts[0]
    decoded, err := base64.StdEncoding.DecodeString(parts[1])
    if err != nil {
        return nil, "", err
    }

    ext, err := mime.ExtensionsByType(mimeType)
    if err != nil {
        return nil, "", err
    }

    return decoded, ext[0], nil
}

```

程式碼結構與 `greeter` 流程非常相似，但這次是改為以獨立的 CLI 應用程式執行。系統不會無限期封鎖，而是會執行 `glowUp` 流程、解碼輸出內容，並將產生的二進位資料儲存到磁碟。

我們需要解碼輸出內容，因為模型會以下列格式傳回圖片資料：

```
data:<mime type>;base64,<base64 encoded image>
```

您可以在 `decode` 函式中看到，我們使用字串操控來分割 MIME 類型和編碼圖片部分。接著，我們使用 `mime.ExtensionByType` 和 `base64.DecodeString` 函式擷取所需資訊，以儲存檔案。

測試還原流程

現在終於可以執行這個流程，並使用真實圖片！如果手邊沒有需要修復的舊照片，可以試試這張從美國國會圖書館網站擷取的公有領域圖片：



以下是相片的直接連結，可傳遞至流程：<https://tile.loc.gov/storage-services/service/pnp/fsa/8c01000/8c01700/8c01765v.jpg>

```
export IMAGE_URL="https://tile.loc.gov/storage-  
services/service/pnp/fsa/8c01000/8c01700/8c01765v.jpg"  
go run main.go --url $IMAGE_URL
```

以下是還原並彩色化的輸出內容：



成功！

6. 將 glowUp 部署為網路服務

如果您想在網路服務中公開流程，而非使用指令列應用程式，Genkit 提供便利的方式，可使用 `genkit.Handler` 介面卡將流程轉換為端點。下列程式碼會使用 `genkit.Handler`，將 `glowUp` 流程註冊為 `glowUp` 端點。

您可以使用標準程式庫中的 `http` 套件，照常透過一般 HTTP 伺服器公開這些函式，但 Genkit 也提供 `server` 外掛程式，可協助處理一些常見的伺服器樣板，例如妥善處理關機信號。

將 `main.go` 的內容替換為下列程式碼，即可建立網路伺服器。

```

package main

import (
    "context"
    "errors"
    "fmt"
    "log"
    "net/http"
    "os"

    "github.com/firebase/genkit/go/ai"
    "github.com/firebase/genkit/go/genkit"
    "github.com/firebase/genkit/go/plugins/googlegenai"
    "github.com/firebase/genkit/go/plugins/server"
)

func main() {
    ctx := context.Background()

    PORT := os.Getenv("PORT")
    if PORT == "" {
        PORT = "8080"
    }

    listenAddr := ":" + PORT

    // Initialize Genkit with the Vertex AI plugin
    g := genkit.Init(ctx,
        genkit.WithPlugins(&googlegenai.VertexAI{}),
    )

    // Input schema for the glowUp flow
    type Input struct {
        URL          string `json:"url,omitempty"`
        ContentType string `json:"contentType,omitempty"`
    }

    genkit.DefineFlow(g, "glowUp", func(ctx context.Context, input Input) (string, error) {
        {
            prompt := genkit.LookupPrompt(g, "glowup")
            if prompt == nil {
                return "", errors.New("prompt 'glowup' not found")
            }

            resp, err := prompt.Execute(ctx, ai.WithInput(input))
            if err != nil {
                return "", fmt.Errorf("generation failed: %w", err)
            }

            return resp.Media(), nil
        }
    })
}

```



```

log.Printf("GlowUp Flow Server started. Listening on %s", listenAddr)
mux := http.NewServeMux()
for _, flow := range genkit.ListFlows(g) {
    mux.HandleFunc("POST /"+flow.Name(), genkit.Handler(flow))
}

if err := server.Start(ctx, listenAddr, mux); err != nil {
    // Check if the error is due to context cancellation
    if ctx.Err() != nil {
        log.Println("GlowUp server shutting down gracefully...")
        return
    }
    log.Fatal(err)
}
}

```

您可以在本機或雲端執行這項服務。現在程式已完成，因此可以直接執行檔案，不必透過 Genkit CLI 啟動。例如：

```
go run main.go
```

由於這是封鎖作業，您需要啟動第二個終端機來測試。最快的方式是使用 `curl` 指令：

```

curl -sS -X POST http://localhost:8080/glowUp \
-H "Content-Type: application/json" \
-d '{"data":{"url": $IMAGE_URL, "contentType":"image/jpeg"}}' \
> result.json

```

由於這是伺服器回應，我們需要解碼 Base64 圖片：

```
cat result.json | jq -r '.result' | awk -F ',' '{print $2}' | base64 -d > restored.png
```

注意：進行這項測試的最佳方式是建立小型用戶端，同時將要求傳送至伺服器並剖析回應，然後將回應儲存至檔案。您可以使用本章稍後提供的提示「氛圍編碼」。

將網路服務部署至 Cloud Run

這個伺服器「在我的電腦上運作」沒問題，但理想情況是將其部署到其他位置，供所有使用者存取。部署這類服務最方便的方式之一，就是使用 Cloud Run 的「從來源部署」功能。有了這項功能，您甚至不必自行建構容器，一切都會自動完成。

注意：如果您是第一次執行 `gcloud run deploy`，系統會要求您啟用 Cloud Run API。

如要使用從來源部署功能將這項服務部署至 Cloud Run，請執行下列指令 (將專案 ID 替換為您自己的 ID)：

```
gcloud run deploy glowup --source . --region us-central1 --no-allow-unauthenticated --set-env-vars
GOOGLE_GENAI_USE_VERTEXAI=True,GOOGLE_CLOUD_LOCATION=$GOOGLE_CLOUD_LOCATION,GOOGLE_CLOUD_PROJECT=$GOOGLE_CLOUD_PROJECT
```

部署作業需要幾分鐘才能完成。完成後，您可以透過 `curl` 傳送另一項要求，測試端點：

```
GLOWUP_URL=$(gcloud run services describe glowup --region us-central1 --format='value(status.url)')

curl -X POST "$GLOWUP_URL/glowUp" \
  -H "Authorization: Bearer $(gcloud auth print-identity-token)" \
  -H "Content-Type: application/json" \
  -d "{\"data\":{\"url\":\"$IMAGE_URL\", \"contentType\":\"image/jpeg\"} }" \
  > result.json
```

同樣地，我們需要解碼 base64 圖片：

```
cat result.json | jq -r '.result' | awk -F ',' '{print $2}' | base64 -d >
restored_cloudrun.png
```

現在我們有可正常運作的相片修復網路伺服器了！

選用：以「直覺式程式開發」方式開發用戶端應用程式

手動編寫程式碼很有趣，但如果您從未做過特定類型的專案，也可能充滿挑戰。幸好，現在有程式設計代理可協助我們加快這個程序。

在這個步驟中，我們不會自行編寫程式碼，而是要求 Gemini CLI (或您偏好的程式設計代理) 為我們完成這項工作。使用下列提示：

GlowUp is a photo restoration service that takes a restoration request as input and returns a restored picture as output. Your task is to create a client application that uses this server to process image urls and save a restored file locally.

TODO:

- Write a CLI application that receives three arguments: an url (required), content type (optional, defaults to image/jpeg) and addr (server address, optional, defaults to localhost:8080)
 - The CLI should send a POST request to the /glowUp endpoint in the server with the body `'{"data":{"url": <url>, "contentType": <contentType>} }'`
 - The server response should be parsed by stripping the "data:" prefix. The remainder will have the format `<mimeType>;base64,<encoded imageData>`
 - Extract the mime type and convert to a file extension using the mime package
 - Extract the image data and decode it using the base64 package
- Save a file named "restored" + the detected file extension

Acceptance Criteria:

- The client builds successfully
- Use the client to restore the image `https://tile.loc.gov/storage-services/service/pnp/fsa/8c01000/8c01700/8c01765v.jpg`
- Check that restored.png exists after processing the image above

由於同一模組中不能有兩個「main」函式，因此代理程式可能需要稍微重整檔案。觀察 Agent 的運作情形，並在必要時提供伺服器程式碼或程式碼片段，引導 Agent 進行正確的實作。

實驗室結束後清除所用資源

注意：請務必在結束本工作階段前刪除 Cloud Run 部署作業，尤其是使用未經驗證的 Cloud Run 執行個體時，以免 GCP 帳單帳戶產生預期外的費用。

步驟 7 · 約 2 分鐘

7. 結論

恭喜！您已成功使用 Genkit 和 Nano Banana Pro，建構高保真相片修復應用程式。

在本程式碼研究室中，您已學會如何：

- 設定環境，開發 Genkit Go 應用程式
- 使用 `dotprompt` 建立多模態提示
- 使用提示範本建立 Genkit 流程
- 使用 Nano Banana Pro 處理圖片
- 將 Genkit 流程封裝為指令列應用程式和網路服務
- 將 Genkit 應用程式部署至 Cloud Run

完成測試後，請記得清理環境。

後續步驟

您可以繼續探索這個平台上的其他程式碼研究室，或是自行改善 glowUp。

如需改善建議，不妨試試：

- 啟用 glowUp CLI 來還原本機檔案
- 為 GlowUp 網路服務建立前端
- 從資料自動偵測 MIME 類型
- 執行其他類型的圖像處理作業 (例如將相片轉換成繪圖、將繪圖轉換成相片、將草圖轉換成最終作品等)。
- 建立或改善用戶端 (或許可以建構智慧型手機應用程式？)

影片內容有無窮的可能性！如果覺得獨自上路有點令人畏懼，隨時可以尋求程式設計代理程式的協助，例如 [Gemini CLI](#) 或 [Antigravity](#)。如要進一步使用 Genkit，搭配 [Genkit MCP 伺服器](#) 的編碼代理程式可讓您事半功倍。

最後，如要存取這個存放區的完整程式碼，請前往[這裡](#)。

祝您編寫程式一切順利！
